

Trong Android, Adapter là một thành phần quan trọng đóng vai trò cầu nối giữa dữ liệu và các View hiển thị dữ liệu đó (như `ListView`, `GridView`, `Spinner`, `RecyclerView`). Có nhiều loại Adapter khác nhau, mỗi loại phù hợp với các trường hợp sử dụng cụ thể.

Các loại Adapter chính trong Android

Có thể phân loại Adapter thành các nhóm chính sau:

1. **ArrayAdapter**: Dùng cho dữ liệu đơn giản (mảng hoặc `ArrayList`) của một kiểu dữ liệu duy nhất (ví dụ: `String`, `Integer`).
2. **SimpleAdapter**: Dùng cho dữ liệu phức tạp hơn, thường là `List<Map<String, ?>>`, nơi mỗi `Map` đại diện cho một hàng dữ liệu và chứa các cặp khóa-giá trị.
3. **CursorAdapter / SimpleCursorAdapter**: Dùng cho dữ liệu lấy từ `Cursor` (thường là từ cơ sở dữ liệu SQLite).
4. **BaseAdapter (Custom Adapter)**: Lớp cơ sở để tạo Adapter tùy chỉnh hoàn toàn, khi các loại Adapter có sẵn không đáp ứng được yêu cầu hiển thị phức tạp.
5. **RecyclerView.Adapter**: Adapter chuyên dụng cho `RecyclerView`, được thiết kế để hiệu quả hơn và linh hoạt hơn `ListView/GridView` truyền thống.

So sánh và Khi nào dùng loại nào

Dưới đây là so sánh chi tiết và hướng dẫn khi nào nên sử dụng từng loại:

1. ArrayAdapter

- **Mô tả**: Là loại Adapter đơn giản nhất, được sử dụng khi nguồn dữ liệu là một mảng hoặc `ArrayList` của các đối tượng cùng kiểu (thường là `String`). Nó có thể hiển thị dữ liệu vào một `TextView` duy nhất trong mỗi hàng.
- **Ưu điểm**:
 - Dễ sử dụng và triển khai cho các trường hợp đơn giản.
 - Tự động xử lý việc chuyển đổi đối tượng thành `String` để hiển thị (bằng cách gọi `toString()` của đối tượng).
- **Nhược điểm**:
 - Chỉ có thể hiển thị một `TextView` trong mỗi hàng mặc định.
 - Không linh hoạt khi cần hiển thị nhiều loại View khác nhau (ví dụ: `ImageView`, nhiều `TextView`) hoặc layout phức tạp cho mỗi hàng.
- **Khi nào dùng**:
 - Hiển thị danh sách các chuỗi đơn giản (ví dụ: danh sách tên, danh sách số).
 - Sử dụng với `Spinner` hoặc `ListView` khi mỗi mục chỉ cần một `TextView`.
 - **Ví dụ trong tài liệu**: Lab `Spinner` sử dụng `ArrayAdapter` để hiển thị danh sách các bảng cửu chương.

Ví dụ:

```
// Dữ liệu
String[] fruits = {"Apple", "Banana", "Orange", "Grape"};

// Tạo ArrayAdapter
ArrayAdapter<String> adapter = new ArrayAdapter<>(
    this, // Context
    android.R.layout.simple_list_item_1, // Layout mặc định của Android cho một TextView
    fruits // Dữ liệu
);

// Gán cho ListView
listView.setAdapter(adapter);
```

2. SimpleAdapter

- **Mô tả:** Được sử dụng khi nguồn dữ liệu là một **List** của các **Map**, trong đó mỗi **Map** đại diện cho một hàng dữ liệu và chứa các cặp khóa-giá trị. **SimpleAdapter** cho phép bạn ánh xạ các giá trị từ **Map** vào các View cụ thể trong layout của mỗi hàng.
- **Ưu điểm:**
 - Linh hoạt hơn **ArrayAdapter** vì có thể hiển thị nhiều loại View (ví dụ: **TextView**, **ImageView**) trong mỗi hàng.
 - Dễ dàng ánh xạ dữ liệu từ **Map** vào các View bằng cách chỉ định khóa và ID của View.
- **Nhược điểm:**
 - Yêu cầu dữ liệu phải được tổ chức dưới dạng **List<Map<String, ?>>**, có thể không tự nhiên cho mọi loại dữ liệu.
 - Không hỗ trợ **ViewHolder Pattern** một cách tự động, có thể dẫn đến hiệu suất kém cho danh sách lớn nếu không được tối ưu hóa thủ công.
- **Khi nào dùng:**
 - Khi bạn có dữ liệu dạng **Map** và muốn hiển thị nhiều thông tin trong mỗi hàng (ví dụ: tên và mô tả).
 - Khi bạn cần ánh xạ dữ liệu từ các cột cụ thể của một nguồn dữ liệu (như JSON) vào các View trong layout.

Ví dụ:

```
// Dữ liệu: List of Maps
List<Map<String, String>> data = new ArrayList<>();
Map<String, String> item1 = new HashMap<>();
item1.put("title", "Item 1");
item1.put("description", "Description for item 1");
data.add(item1);

Map<String, String> item2 = new HashMap<>();
item2.put("title", "Item 2");
item2.put("description", "Description for item 2");
data.add(item2);

// Layout cho mỗi hàng (my_list_item.xml)
// <LinearLayout ...>
//     <TextView android:id="@+id/title_text" ... />
//     <TextView android:id="@+id/description_text" ... />
// </LinearLayout>

// Khóa từ Map
String[] from = {"title", "description"};
// ID của các View trong layout hàng
int[] to = {R.id.title_text, R.id.description_text};

// Tạo SimpleAdapter
SimpleAdapter adapter = new SimpleAdapter(
    this, // Context
    data, // Dữ liệu
    R.layout.my_list_item, // Layout cho mỗi hàng
    from, // Khóa từ Map
    to // ID của các View
);

// Gán cho ListView
listView.setAdapter(adapter);
```

3. CursorAdapter / SimpleCursorAdapter

- **Mô tả:** Được thiết kế đặc biệt để hiển thị dữ liệu từ một **Cursor** (thường là kết quả của một truy vấn cơ sở dữ liệu SQLite).
 - **CursorAdapter**: Là lớp cơ sở, yêu cầu bạn tự định nghĩa cách ánh xạ dữ liệu từ **Cursor** vào các View.
 - **SimpleCursorAdapter**: Là một triển khai của **CursorAdapter** giúp đơn giản hóa việc ánh xạ dữ liệu từ các cột của **Cursor** vào các View cụ thể trong layout.

- **Ưu điểm:**
 - Hiệu quả khi làm việc với dữ liệu từ cơ sở dữ liệu, vì nó không cần tải toàn bộ dữ liệu vào bộ nhớ cùng một lúc.
 - Tự động cập nhật UI khi dữ liệu trong `Cursor` thay đổi (khi `swapCursor()` được gọi).
- **Nhược điểm:**
 - Chỉ hoạt động với dữ liệu từ `Cursor`.
 - `SimpleCursorAdapter` vẫn có những hạn chế về tùy chỉnh so với `BaseAdapter`.
- **Khi nào dùng:**
 - Khi nguồn dữ liệu của bạn là từ cơ sở dữ liệu SQLite và bạn muốn hiển thị nó trong `ListView` hoặc `GridView`.
 - **Ví dụ trong tài liệu:** Lab `SQLite` có thể mở rộng để sử dụng `CursorAdapter` nếu bạn muốn hiển thị danh sách user trong `ListView` trực tiếp từ `Cursor`.

Ví dụ (`SimpleCursorAdapter`):

```
// Giả sử bạn có một Cursor từ truy vấn DB
Cursor cursor = myDatabaseHelper.getAllUsersCursor();

// Khóa từ Cursor (tên cột)
String[] fromColumns = {"username", "hoten"};
// ID của các View trong layout hàng
int[] toViews = {R.id.username_text, R.id.hoten_text};

// Tạo SimpleCursorAdapter
SimpleCursorAdapter adapter = new SimpleCursorAdapter(
    this, // Context
    R.layout.my_user_item, // Layout cho mỗi hàng
    cursor, // Cursor chứa dữ liệu
    fromColumns, // Tên cột từ Cursor
    toViews, // ID của các View
    0 // Flags (0 hoặc CursorAdapter.FLAG_REGISTER_CONTENT_OBSERVER)
);

// Gán cho ListView
listView.setAdapter(adapter);

// Khi dữ liệu thay đổi, bạn có thể gọi:
// adapter.changeCursor(newCursor);
```

4. `BaseAdapter` (Custom Adapter)

- **Mô tả:** Là lớp trừu tượng cơ sở cho tất cả các Adapter. Khi bạn cần kiểm soát hoàn toàn cách dữ liệu được hiển thị trong mỗi hàng (ví dụ: layout phức tạp, nhiều loại View, xử lý sự kiện riêng cho từng View con), bạn sẽ tạo một lớp Adapter tùy chỉnh bằng cách kế thừa từ `BaseAdapter` và ghi đè các phương thức cần thiết (`getCount()`, `getItem()`, `getItemId()`, `getView()`).
- **Ưu điểm:**
 - Cung cấp sự linh hoạt và tùy chỉnh tối đa.
 - Cho phép bạn triển khai `ViewHolder Pattern` để tối ưu hóa hiệu suất cuộn.
- **Nhược điểm:**
 - Yêu cầu nhiều code hơn để triển khai so với các Adapter có sẵn.
- **Khi nào dùng:**
 - Khi bạn cần hiển thị dữ liệu phức tạp với layout tùy chỉnh cho mỗi hàng.
 - Khi bạn cần tối ưu hóa hiệu suất cuộn cho danh sách lớn bằng cách sử dụng `ViewHolder Pattern`.
 - Khi bạn cần xử lý các sự kiện trên các View con trong mỗi hàng.
 - **Ví dụ trong tài liệu:** Lab `GridView` sử dụng một `Custom Adapter` kế thừa từ `BaseAdapter` để hiển thị logo và tên.

Ví dụ (cấu trúc cơ bản):

```

public class MyCustomAdapter extends BaseAdapter {
    private Context context;
    private List<MyObject> dataList; // Dữ liệu của bạn

    public MyCustomAdapter(Context context, List<MyObject> dataList) {
        this.context = context;
        this.dataList = dataList;
    }

    @Override
    public int getCount() {
        return dataList.size();
    }

    @Override
    public Object getItem(int position) {
        return dataList.get(position);
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        ViewHolder holder;
        if (convertView == null) {
            convertView = LayoutInflater.from(context).inflate(R.layout.my_custom_item, parent,
false);
            holder = new ViewHolder();
            holder.textView = convertView.findViewById(R.id.my_text_view);
            holder.imageView = convertView.findViewById(R.id.my_image_view);
            convertView.setTag(holder);
        } else {
            holder = (ViewHolder) convertView.getTag();
        }

        MyObject item = dataList.get(position);
        holder.textView.setText(item.getName());
        holder.imageView.setImageResource(item.getImageResId());

        return convertView;
    }

    static class ViewHolder {
        TextView textView;
        ImageView imageView;
    }
}

```

5. RecyclerView.Adapter

- **Mô tả:** Là Adapter hiện đại và được khuyến nghị cho **RecyclerView**. **RecyclerView** là một phiên bản nâng cao và linh hoạt hơn của **ListView/GridView**, được thiết kế để xử lý danh sách lớn và phức tạp một cách hiệu quả hơn. **RecyclerView.Adapter** hoạt động cùng với **RecyclerView.ViewHolder** và **LayoutManager**.
- **Ưu điểm:**
 - **Hiệu suất cao:** Tự động triển khai **ViewHolder Pattern** và tái chế View hiệu quả.

- **Linh hoạt:** Tách biệt việc quản lý layout (**LayoutManager**) và hoạt ảnh (**ItemAnimator**) khỏi Adapter, cho phép tùy chỉnh dễ dàng hơn.
- **Mạnh mẽ:** Hỗ trợ nhiều loại View trong cùng một danh sách, hoạt ảnh thêm/xóa/cập nhật item.
- **Nhược điểm:**
 - Yêu cầu nhiều code boilerplate hơn một chút so với **ArrayAdapter** hoặc **SimpleAdapter** cho các trường hợp đơn giản nhất.
 - Cần hiểu rõ hơn về **ViewHolder** và **LayoutManager**.
- **Khi nào dùng:**
 - **Luôn luôn ưu tiên sử dụng **RecyclerView** và **RecyclerView.Adapter**** cho các danh sách trong ứng dụng Android mới.
 - Khi bạn cần hiển thị danh sách dữ liệu động, có thể thay đổi, thêm, xóa item.
 - Khi bạn cần hiệu suất cuộn mượt mà cho danh sách lớn.
 - Khi bạn cần tùy chỉnh layout của từng item hoặc có nhiều loại item khác nhau trong cùng một danh sách.
 - **Ví dụ trong tài liệu:** Ứng dụng quản lý User sẽ sử dụng **RecyclerView** và **RecyclerView.Adapter** để hiển thị danh sách người dùng.

Ví dụ (cấu trúc cơ bản):

```
public class MyRecyclerViewAdapter extends
RecyclerView.Adapter<MyRecyclerViewAdapter.MyViewHolder> {

    private List<MyObject> dataList;
    private OnItemClickListener listener;

    public interface OnItemClickListener {
        void onItemClick(MyObject item);
    }

    public MyRecyclerViewAdapter(List<MyObject> dataList, OnItemClickListener listener) {
        this.dataList = dataList;
        this.listener = listener;
    }

    @NonNull
    @Override
    public MyViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.my_recycler_item,
parent, false);
        return new MyViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull MyViewHolder holder, int position) {
        MyObject item = dataList.get(position);
        holder.textView.setText(item.getName());
        holder.imageView.setImageResource(item.getImageResId());
        holder.itemView.setOnClickListener(v -> listener.onItemClick(item));
    }

    @Override
    public int getItemCount() {
        return dataList.size();
    }

    // ViewHolder class
    static class MyViewHolder extends RecyclerView.ViewHolder {
        TextView textView;
        ImageView imageView;
    }
}
```

```
MyViewHolder(@NonNull View itemView) {
    super(itemView);
    textView = itemView.findViewById(R.id.my_text_view);
    imageView = itemView.findViewById(R.id.my_image_view);
}
}
```

Tóm tắt so sánh:

Đặc điểm	ArrayAdapter	SimpleAdapter	CursorAdapter / SimpleCursorAdapter	BaseAdapter (Custom)	RecyclerView.Adapter
Nguồn dữ liệu	Array, ArrayList của đối tượng đơn giản	List<Map<String, ?>>	Cursor (từ DB)	Bất kỳ cấu trúc dữ liệu nào	Bất kỳ cấu trúc dữ liệu nào
Độ phức tạp tap UI	Đơn giản (1 TextView)	Trung bình (nhiều View với ánh xạ khóa-ID)	Trung bình (nhiều View với ánh xạ cột-ID)	Cao (layout tùy chỉnh hoàn toàn)	Cao (layout tùy chỉnh hoàn toàn)
Hiệu suất	Tốt cho danh sách nhỏ	Trung bình (cần tối ưu hóa thủ công)	Tốt (tái sử dụng View)	Tốt (với ViewHolder Pattern)	Rất tốt (tự động ViewHolder, tái chế View)
Linh hoạt	Thấp	Trung bình	Trung bình	Cao	Rất cao
Khuyến nghị	Danh sách rất đơn giản, Spinner	Dữ liệu dạng Map, cần nhiều View đơn giản	Dữ liệu từ SQLite DB (cho ListView/GridView)	Khi cần tùy chỉnh cao cho ListView/GridView	Luôn ưu tiên cho danh sách mới

Kết luận:

Trong phát triển ứng dụng Android hiện đại, **RecyclerView và RecyclerView.Adapter là lựa chọn hàng đầu** cho hầu hết các trường hợp hiển thị danh sách. Chúng cung cấp hiệu suất, linh hoạt và khả năng mở rộng vượt trội so với **ListView/GridView** truyền thống và các Adapter đi kèm. Tuy nhiên, việc hiểu rõ các loại Adapter khác vẫn quan trọng để xử lý các dự án cũ hoặc các trường hợp đặc biệt.